

ECE551: Digital System Design & Synthesis

Midterm

Name: _____ **Solutions**

*Closed book except for one 8½ x 11 sheet. No calculators or electronics devices.
All work is to be your own - **show your work** for maximum partial credit.*

1) (5pts) True or False. Fill in the circle completely (●, not ○ or ⊗).

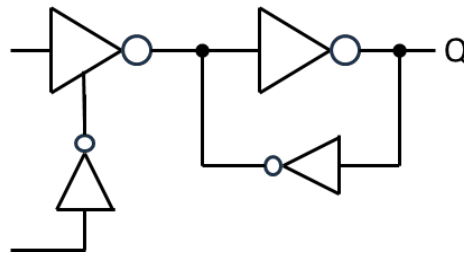
- a. ☒ True ☐ False Many registers involved in datapath do not need to be reset because your design will initialize them before using their contents
- b. ☒ True ☐ False If I have a block that starts out: “**always @(posedge clk, ...)**” I don’t have to be concerned about unintentionally inferring latches in that block.
- c. ☐ True ☒ False Synthesis uses delay #’s specified in verilog during the mapping process (*mapping of logic equations to standard cells in library*) to try to achieve the specified delay.
- d. ☐ True ☒ False In **case** statements, a default case is always needed to avoid unintended latches.
- e. ☐ True ☒ False Using a Moore machine instead of Mealy will ensure that my circuit output(s) can’t glitch.

2) (3pts) Multiple Choice. Select (●, not ○ or ⊗) the most correct choice.

- a. A flip-flop can be described using:
 - ☐ structural Verilog
 - ☐ dataflow Verilog
 - ☐ behavioral Verilog
 - ☒ any of the above

- b. The circuit shown is best described as:

- ☐ Negedge flop
- ☐ Posedge flop
- ☒ Enable low latch
- ☐ Enable high latch
- ☐ RAM cell



- c. In the code shown below what type of flop timing is being modeled?

- ☐ Setup time
- ☒ Clock to Q delay
- ☐ Hold time

```
always @(posedge clk)
  if (rst_n) q <= #(1.5) 1'b0;
  else q <= #(2.1,1.5) d;
```

3) (8pts) Select all that applies.

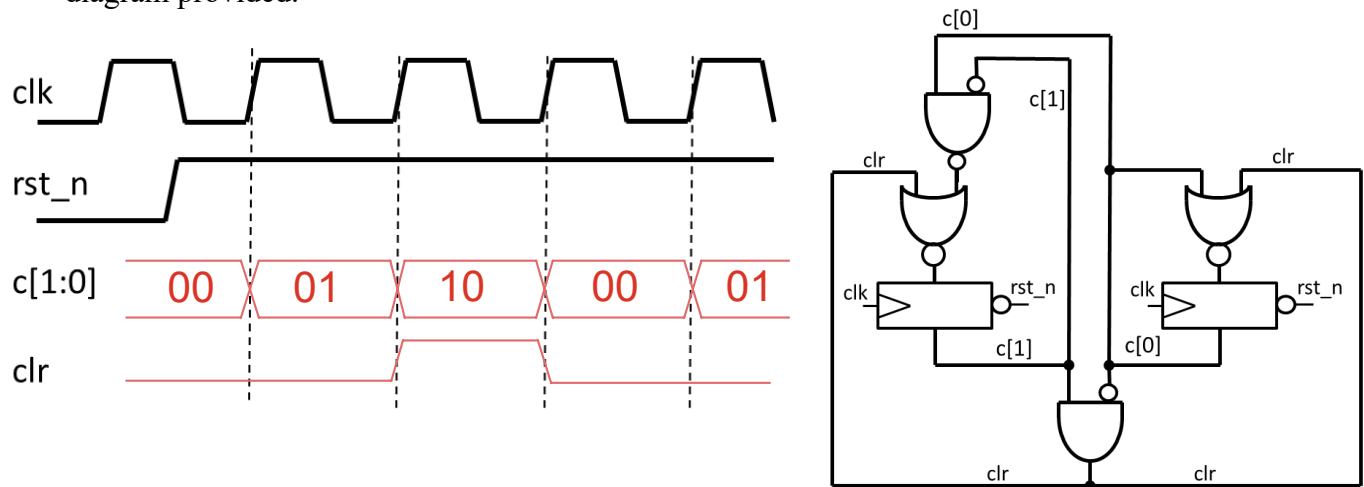
- a. Meta-stability:
- ☐ Occurs when alpha particles impart charge on a node of a latch or flop
 - ☒ Is a hazard to be aware of for a UART receiver design
 - ☒ Can occur when the input of a flop changes coincident with the clock
 - ☐ Can be solved by using a high gain inverter.
- b. Which of the following code will generate a working clock signal?
- ☐ always @(clk1) clk1 = #10 ~clk1;
 - ☒ always @(clk2) clk2 <= #10 ~clk2;
 - ☒ always #10 clk3 <= ~clk3;
 - ☐ always clk4 <= #10 ~clk4;
- c. For the code shown, select all statements that are TRUE:
- ```
always @(a or b) begin
 sum = a ^ b ^ cin;
end
```
- ☐ The code will fail to compile and generate a syntax error because `cin` is used on the right-hand side but omitted from the sensitivity list.
  - ☒ In a simulator, if `a` and `b` are held constant and `cin` toggles, `sum` will fail to update and will improperly hold its previous value.
  - ☒ Replacing the first line with `always_comb begin` is the standard method to fix this block, ensuring the simulator correctly models the synthesized 3-input XOR gate.
- d. When writing synthesis constraints for an input pin of your DUT. Which of the following things do you need to constrain?
- ☐ The rise/fall time of this input
  - ☒ The valid time (arrival time) of the input relative to system `clk`
  - ☐ The input load (capacitance) seen at this pin
  - ☒ The driving strength of this input
  - ☐ The bit-width of this input (in case it is a vector)

**4) (4pts) In the following `casez` block of code, what is displayed when `sel` is 0, 1, x and z?**

```
always@(sel)
 casez(sel)
 1'b0 : $display("Output 0");
 1'b1 : $display("Output 1");
 1'bx : $display("Output x");
 1'bz : $display("Output z");
 endcase
```

| sel    | 0 | 1 | x | z |
|--------|---|---|---|---|
| Output | 0 | 1 | x | 0 |

- 5) (5pts) Analyze the circuit shown and draw in the values of **c[1:0]** and **clr** vs time on the timing diagram provided.



(6pts) For the circuit shown above finish the **always\_ff** and **assign** statements needed to implement it. You should be coding like a rebel engineer, and thinking about the **behavior** you discovered when completing the timing diagram.

```
always_ff @(posedge clk, negedge rst_n)
 if (!rst_n)
 c <= 2'b00;
 else if (clr)
 c <= 2'b00;
 else
 c <= c + 1;
 assign clr = (c == 2'b10);
```

- 6) **UART** design. Your system runs on a 30MHz clock. You are to transmit data over a **UART** with a Baud rate of 10,000. How many bits wide does the baud counter need to be? (2pts)

$$30 \times 10^6 / 10 \times 10^3 = 3000$$

Answer:

12

(4pts) Write the dataflow statement used to determine when your UART shift register should **shift**.

```
assign shift = (baud_cnt == 12'd3000);
```

(6pts) Write an **always\_ff** block to implement a the 9-bit **shift\_reg** of the UART. Assume control signals of **load** and **shift**.

```
always_ff @(posedge clk, negedge rst_n)
 if (!rst_n)
 shift_reg <= 9'h1FF;
 else if (load)
 shift_reg <= {tx_data, 1'b0};
 else if (shift)
 shift_reg <= {1'b1, shift_reg[8:1]};
```

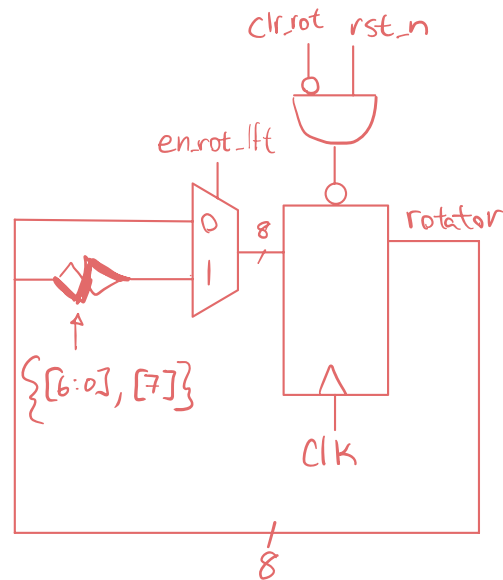
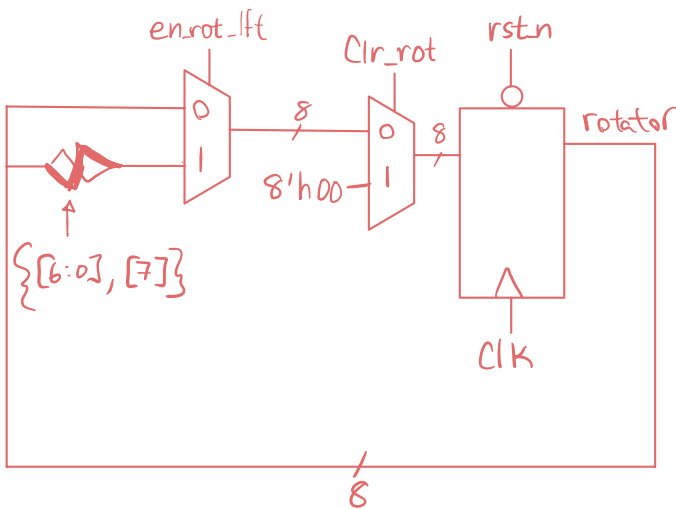
7) You need an 8-bit rotator with a clear and an enable that will be controlled from a state machine.

```
always@(posedge clk, negedge rst_n)
 if (!rst_n)
 rotator <= 8'h00;
 else if (clr_rot)
 rotator <= 8'h00;
 else if (en_rot_lft)
 rotator <= {rotator[6:0], rotator[7]};
endcase
```

```
assign clr_n = ~clr_rot & rst_n;

always@(posedge clk, negedge clr_n)
 if (!clr_n)
 rotator <= 8'h00;
 else if (en_rot_lft)
 rotator <= {rotator[6:0], rotator[7]};
endcase
```

Draw how each will synthesize. (5pts each)



(3pts) Which one would you choose and why?

Choose the left one.

In the one on the right, `clr_n` is formed from combinational logic. you should not tie this to the async rest of FFs since glitches can occur and change the state of the FF.

- 8) You are given a combinational module with the following interface:
- ```
module parity (input [6:0] in, output odd_parity);
```

This module computes the odd parity, which returns 1 if the number of bits set in `in` is odd.

(2pts) How many total input combinations must be tested to make the testbench exhaustive?

Answer: $2^7 = 128$

(15pts) Write a **self-checking** testbench that **exhaustively** tests all input combinations.

```
module parity_tb();

logic [6:0] in;
logic odd_parity;

// instantiate the DUT
parity iDUT(.in(in), .odd_parity(odd_parity));

integer i; // or logic [7:0] i (need 8 bits to count to 128)
initial begin
    for (i = 0; i < 128; i = i + 1) begin
        in = i[6:0];    // assign input
        #1;             // wait for output to stabilize

        // check if output matches expected value. Need:
        // - model to compute expected parity
        // - display error message if mismatch
        if (odd_parity != (^in))
            $display("Test failed for input %b: expected odd_parity = %b, got %b", in, ^in, odd_parity);
    end
    $display("All tests completed.");
end

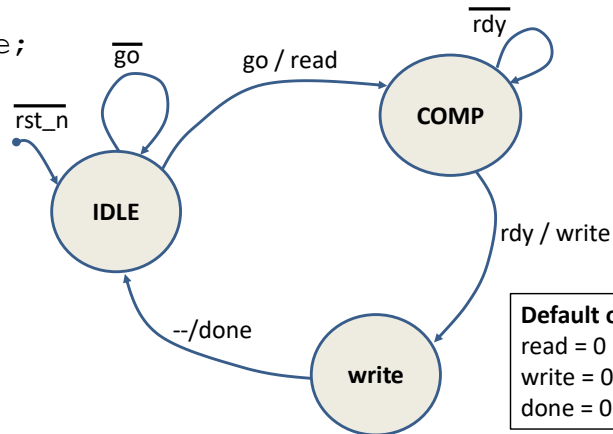
endmodule
```

9) (15pts) Complete the Verilog code for the following FSM bubble diagram.

```
module FSM(clk, rst_n, go, read, write, done);
```

```
input clk, rst_n, go;
```

```
output logic read, write, done;
```



```
// state encoding and declaration
```

```
typedef enum logic [1:0] {IDLE, COMP, WRITE} state_t;
```

```
state_t state, nxt_state;
```

```
// state FF
```

```
always_ff @(posedge clk, negedge rst_n)
```

```
    if (!rst_n)
```

```
        state <= IDLE;
```

```
    else
```

```
        state <= nxt_state;
```

```
// output logic and next state logic
```

```
always_comb begin
```

```
    // default values
```

```
    read = 0;
```

```
    write = 0;
```

```
    done = 0;
```

```
    nxt_state = state;
```

```
    case (state)
```

```
        IDLE: if (go) begin
```

```
            read = 1;
```

```
            nxt_state = COMP;
```

```
        end
```

```
        COMP: if (rdy) begin
```

```
            write = 1;
```

```
            nxt_state = WRITE;
```

```
        end
```

```
        WRITE: begin
```

```
            done = 1;
```

```
            nxt_state = IDLE;
```

```
        end
```

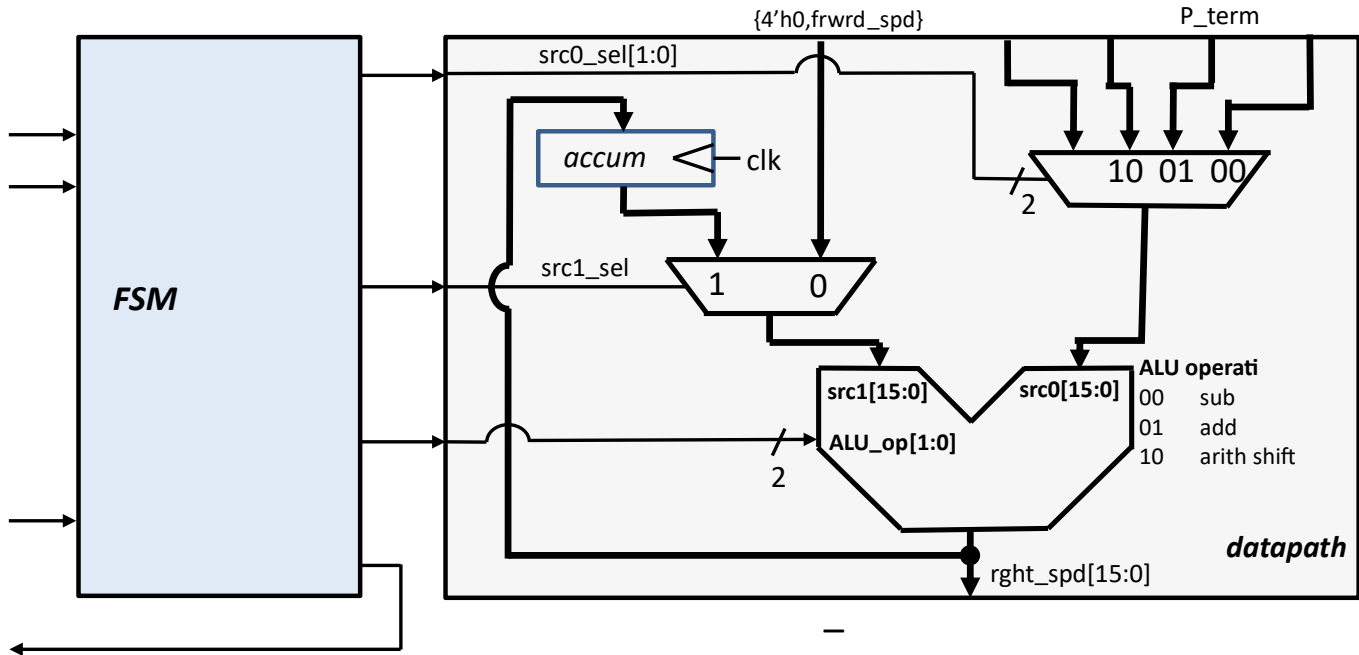
```
        default: nxt_state = IDLE;
```

```
    endcase
```

```
end
```

```
endmodule
```

- 10) We wrote dataflow Verilog to infer all the necessary math for PID in parallel (*computed in a single clock cycle*). One could imagine using a more generic ALU based datapath and performing PID math over several clock cycles. This ALU based datapath would be under the control of a state machine.



You are to design the state machine to control the datapath to produce the calculation:

$$\text{rgh_t_spd} = \text{frwr_d_spd} - (\text{P_term} + \text{I_term} + \text{D_term}) / 8$$

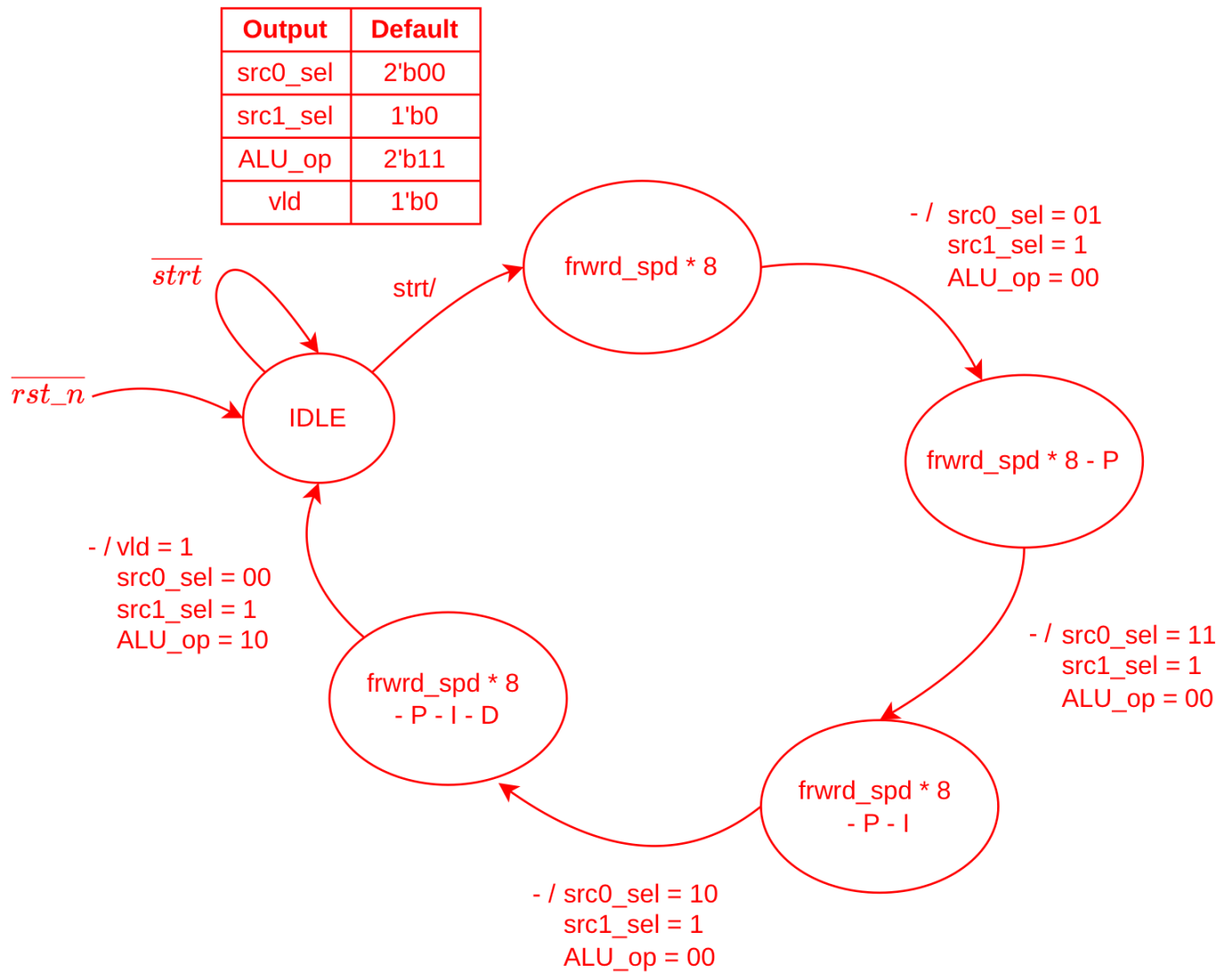
(Hint: You may want to rearrange the equation)

You start the calculation when **strt** is asserted. **vld** should be asserted for the clock cycle that the result (**rgh_t_spd[15:0]**) is valid. When the calculation is finished you should return to the state waiting for **strt** to be asserted again.

On the next page draw the bubble diagram for this SM using proper Mealy notation.

SM Interface:

Signal:	Dir:	Description:
clk, rst_n	in	System clk and active low asynch reset
strt	in	Signal from “the boss” telling you to start the calculation
src0_sel	out	2-bit selector selecting data to feed the ALU on the 16-bit src0 bus
src1_sel	out	Selects accumulator register or frwr_d_spd to feed src1 of ALU
ALU_op[1:0]	out	Selects ALU operation: 00→src1 – src0, 01→src1 + src0, 10→src1 >>> src0, 11→src1 << src0
vld	Out	Back to “the boss” telling them rgh_t_spd[15:0] contains valid results this clk

(12pts) Draw the bubble diagram:

Subtracting P, I, D can be in any order

This page is intentionally left blank.